LUCKY TRADERS

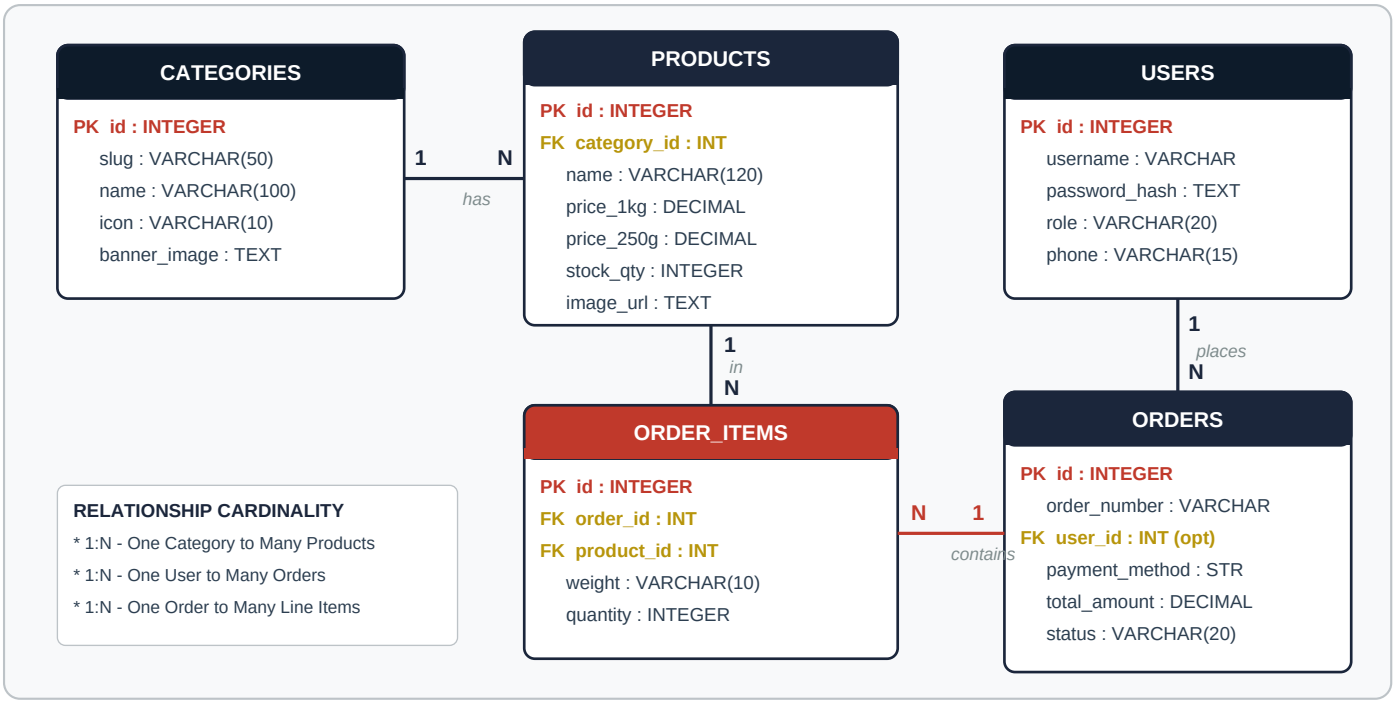
Unified Modeling Language (UML) & System Architectural Specification

Course / Syllabus:	SPPU Final Year Computer Engineering - Project CS331FP
Project Title:	Lucky Traders - Authentic Spices, Chutneys & Pickles Web Portal
Architecture:	3-Tier Web Architecture (Client UI + Flask WSGI + SQLite RDBMS)
Backend Runtime:	Python 3.13 / Flask 3.1 RESTful Web Framework
Database:	SQLite3 Relational Engine (lucky_traders.db) with ACID Transactions
Payment Gateway:	Ashwini Bramhankar UPI (ashutb1210@okhdfcbank) & Cash on Delivery
Documentation Scope:	Complete UML 2.5 Specification (8 Structural & Behavioral Diagrams)
Verification:	Automated Python unittest Suite (8/8 Verified Tests Passing)

Executive Architectural Summary:
This document contains the official architectural design and UML diagrams for the Lucky Traders e-commerce system as required by the University Project Curriculum. The system implements a full-stack, responsive web application with dual packaging selection (1 Kg vs 250 Gm), atomic inventory auto-decrement, dynamic itemized GST bill computation, role-based authentication (Customer vs Administrator), and direct UPI merchant payment integration. All 8 diagrams adhere to formal UML 2.5 standards and accurately model the physical implementation in Python and SQLite.

0. Entity-Relationship (ER) Diagram

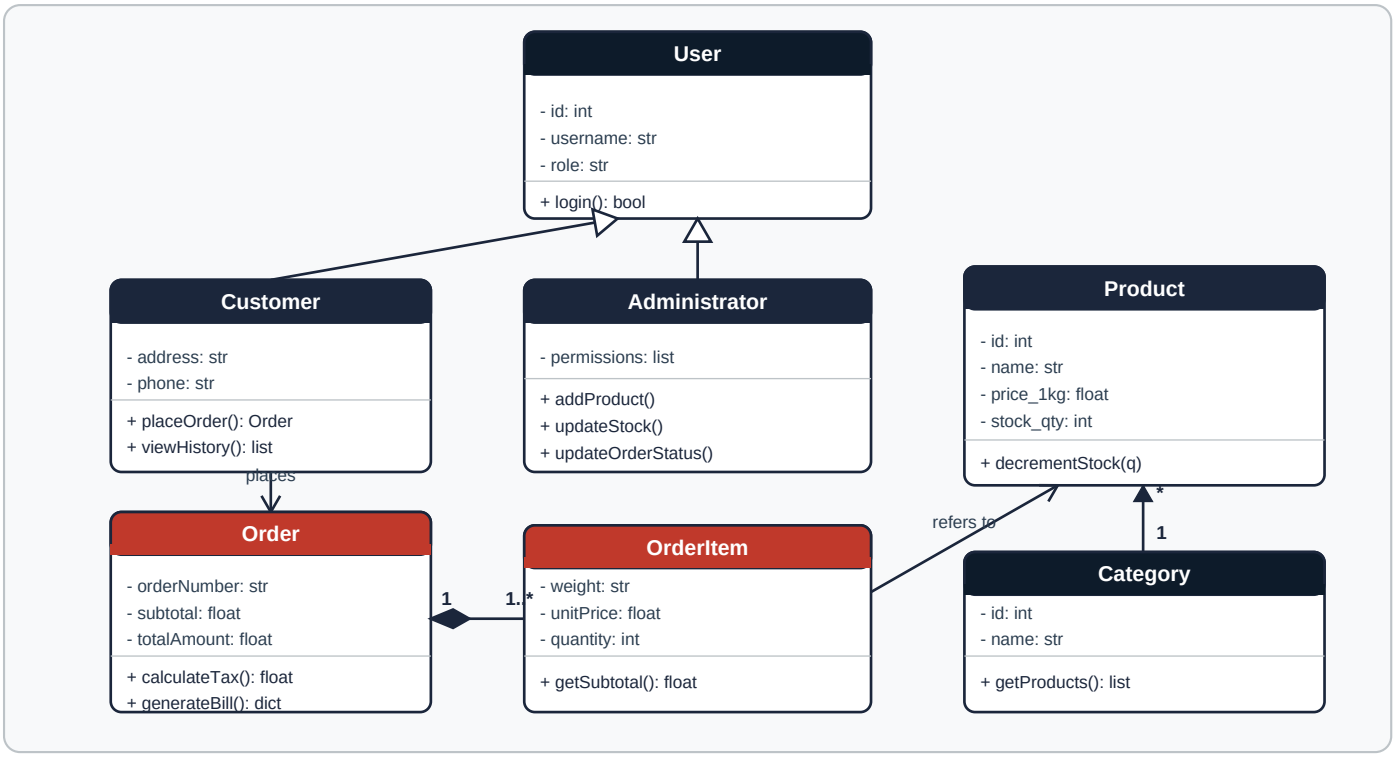
The Entity-Relationship Diagram models the persistent relational data structure of the SQLite database (lucky_traders.db). It captures entities, primary/foreign keys, and relational cardinalities (1:N and M:N).



Entity	Primary Key	Foreign Keys	Cardinality & Description
USERS	id (INTEGER)	None	1:N with ORDERS. Stores customer and administrator credentials.
CATEGORIES	id (INTEGER)	None	1:N with PRODUCTS. Hierarchical grouping (Masalas, Chutneys, Pickles).
PRODUCTS	id (INTEGER)	category_id -> CATEGORIES.id	1:N with ORDER_ITEMS. Stores pricing for 1kg and 250g, and stock_qty.
ORDERS	id (INTEGER)	user_id -> USERS.id (optional)	1:N with ORDER_ITEMS. Contains subtotal, 2.5% CGST, 2.5% SGST, total.
ORDER_ITEMS	id (INTEGER)	order_id -> ORDERS.id, product_id -> PRODUCTS.id	Associative table resolving M:N between ORDERS and PRODUCTS.

1. Class Diagram

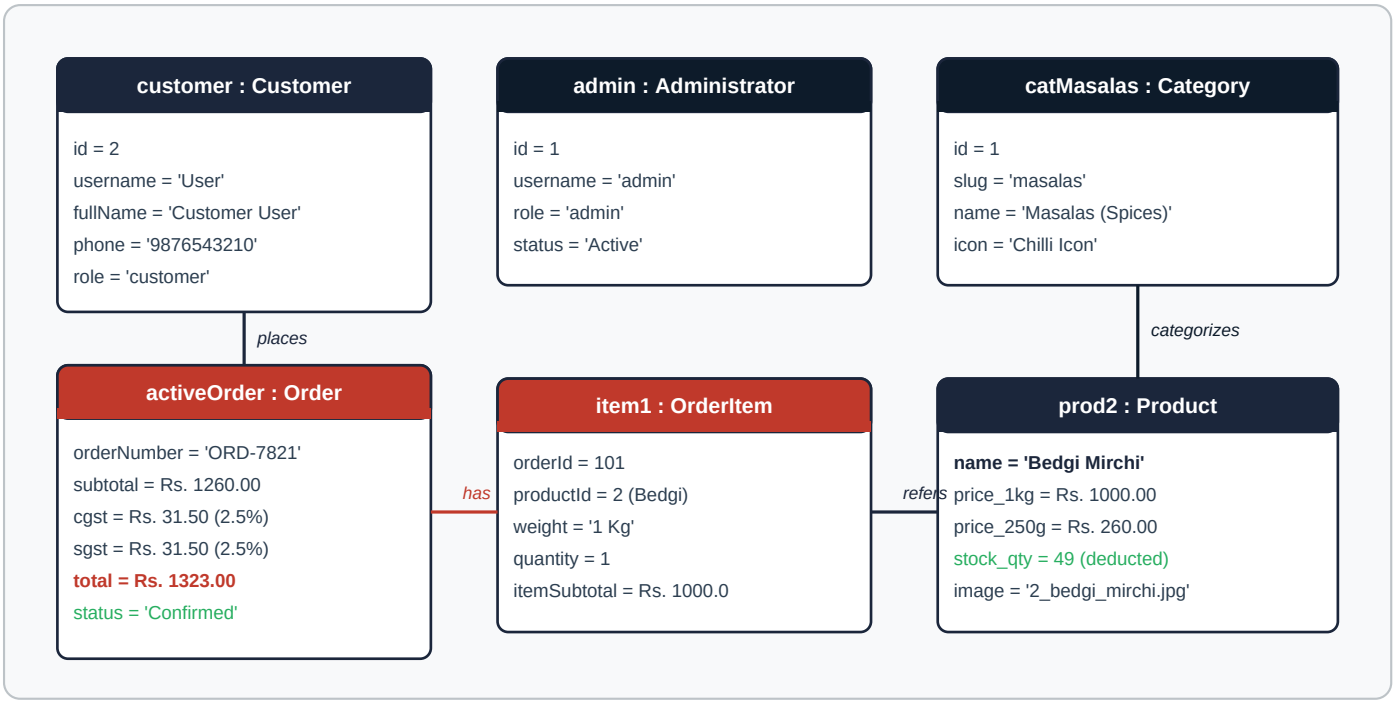
The Class Diagram specifies the object-oriented structure of the backend application services, domain entities, and data access controllers. It illustrates inheritance, composition, and associations.



Design Pattern & OOP Principles:
* **Generalization:** Customer and Administrator inherit from the base User class.
* **Composition:** Order strongly composes OrderItem (1 to 1..*); line items cannot exist without an order.
* **Aggregation:** Category aggregates Product instances (1 to *).
* **Encapsulation:** Stock decrement is encapsulated in Product.decrementStock() and executed atomically in SQL.

2. Object Diagram

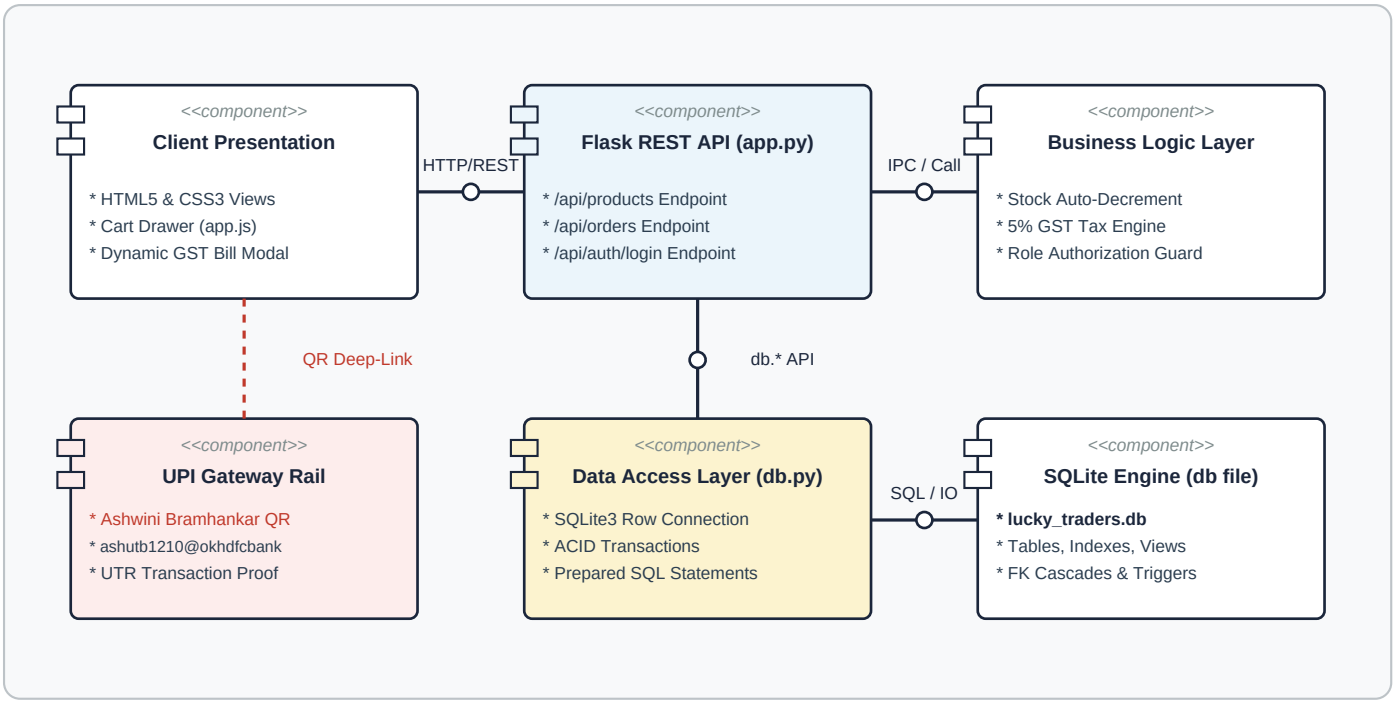
The Object Diagram represents a concrete runtime snapshot of the system during an active customer transaction. It depicts instantiated objects, their bound attribute values, and execution links.



Runtime State Analysis (Order ORD-7821 Execution Snapshot):
* customer:Customer (id=2, 'User') has authenticated and placed activeOrder:Order.
* The order references item1:OrderItem containing 1 unit of prod2:Product (Bedgi Mirchi Powder, 1 Kg, Rs. 1000).
* Total tax is computed at 5% (CGST: Rs. 31.50, SGST: Rs. 31.50), resulting in Rs. 1323.00.
* Product stock is atomically decremented from 50 to 49 upon confirmation.

3. Component Diagram

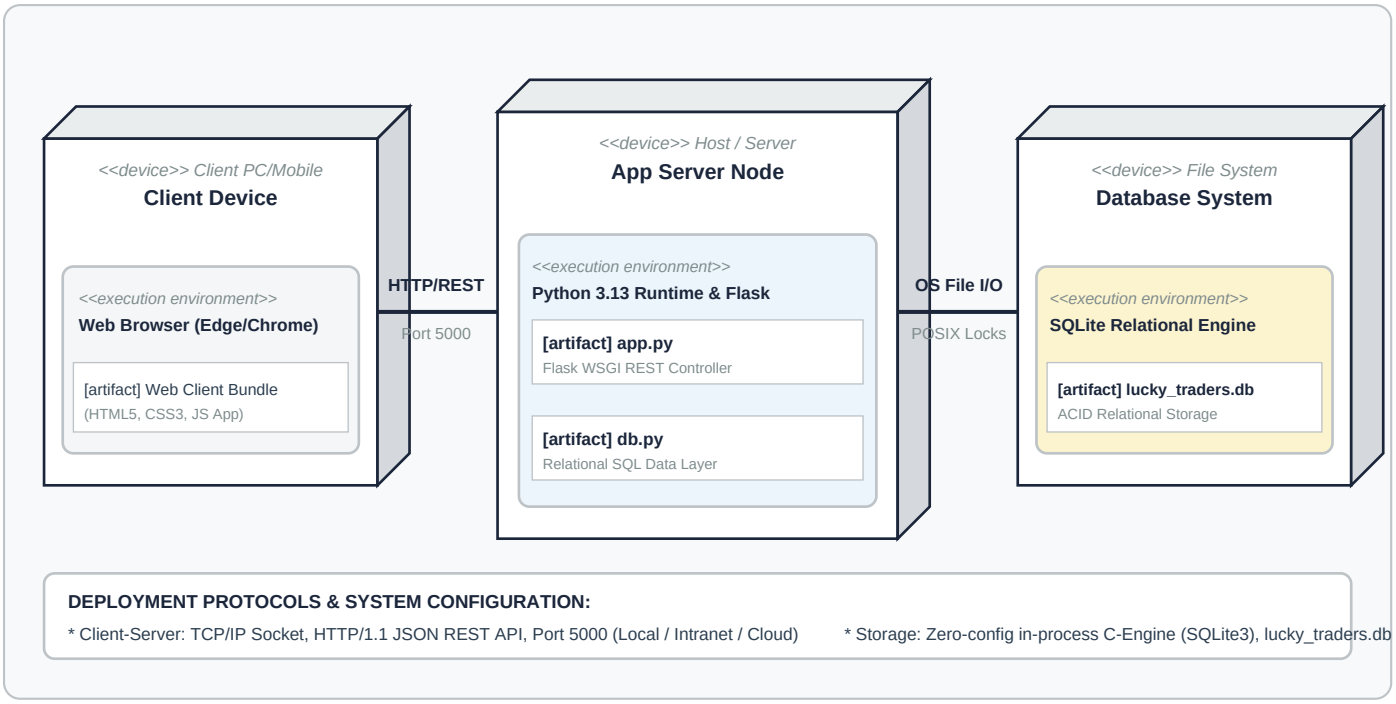
The Component Diagram models the high-level modular software architecture, provided/required interfaces, and physical software components comprising the client and server.



Component	Type	Interface Provided / Consumed	Responsibility
Client Presentation	Frontend Client	Consumes /api/* REST Endpoints	UI rendering, cart state, dynamic invoice printing, modal handling.
Flask REST API (app.py)	Web Controller	Provides HTTP RESTful Endpoints	Route dispatching, CORS, payload validation, status response codes.
Business Logic Layer	Service Domain	Stock & Tax Computation APIs	Auto-decrementing stock, 5% GST computation, role authorization.
Data Access Layer (db.py)	Database Client	Provides db.* Python Functions	Connection pooling, row factory, prepared statements, ACID transactions.
SQLite Storage	Relational Engine	Consumes OS File I/O & Locks	ACID compliant disk storage (lucky_traders.db).

4. Deployment Diagram

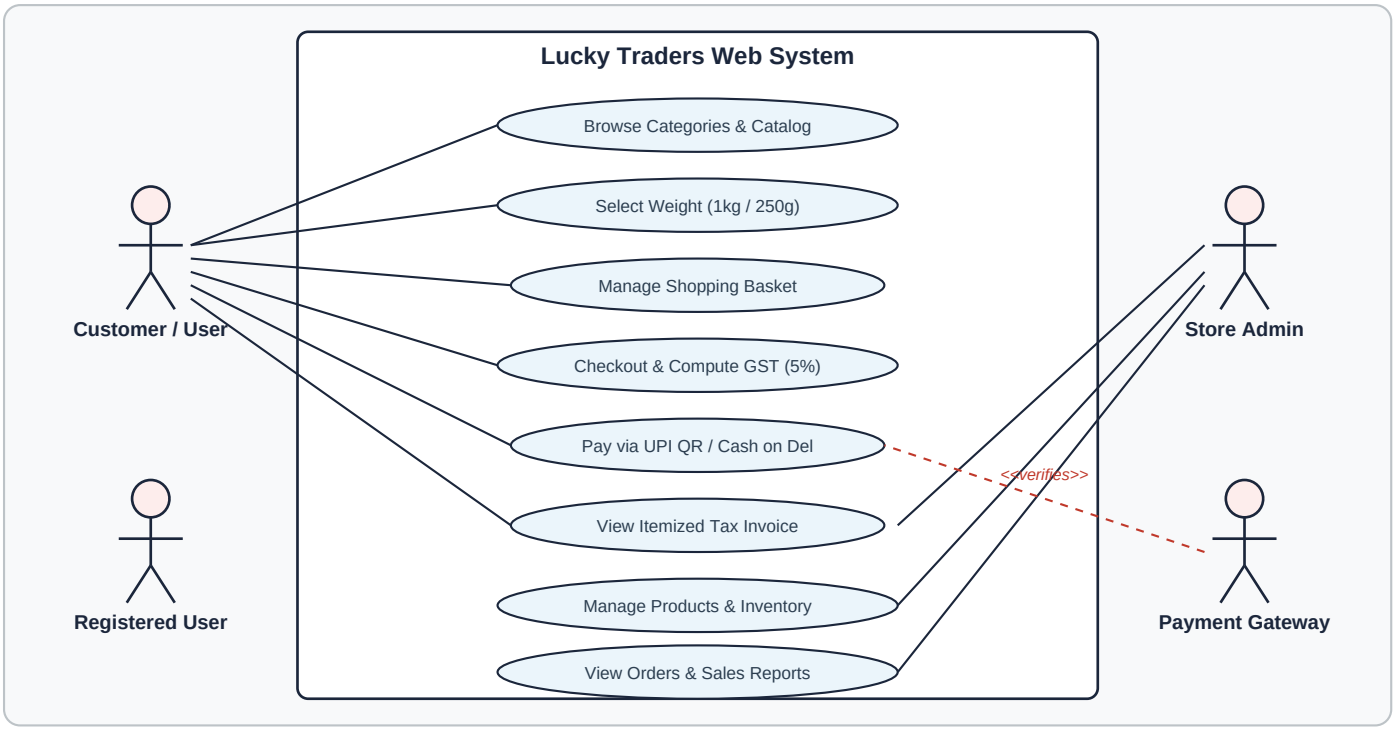
The Deployment Diagram models the physical hardware and execution environment nodes, runtime artifacts, and network protocols required to execute Lucky Traders in development and production.



Deployment Topology & Node Specifications:
* **Client Device Node:** Any modern web browser running on Windows/Linux/macOS or mobile devices. Communicates via HTTP/1.1 or HTTP/2 over TCP/IP.
* **Application Server Node:** Python 3.13 runtime hosting Flask WSGI server on port 5000. Deploys app.py and db.py.
* **Database System:** SQLite embedded in-process database engine writing to local disk file lucky_traders.db with POSIX/Win32 file locks.

5. Use Case Diagram

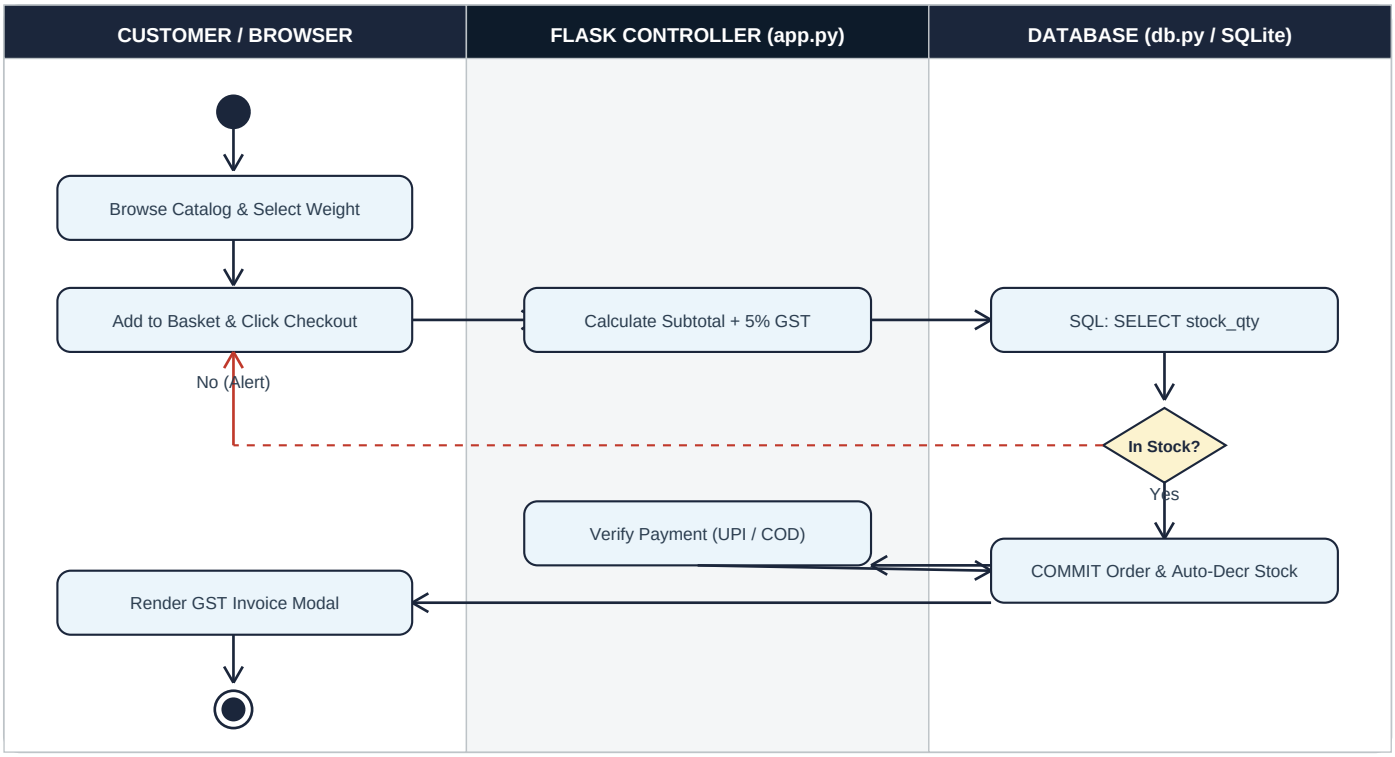
The Use Case Diagram defines functional requirements from the perspective of primary actors (Customer, Administrator, and Payment Gateway), showing system boundary and use case relationships.



Actor	Role Classification	Associated Key Use Cases
Customer (Guest / User)	Primary External Actor	Browse catalog, select packaging (1kg/250g), manage cart, checkout, scan UPI QR, print bill.
Store Administrator	Primary Internal Actor	Manage catalog products, adjust stock quantities, view orders, update dispatch status, view reports.
Payment Gateway (UPI)	Secondary External Actor	Receive deep-link intent, authenticate banking PIN, return transaction UTR confirmation.

6. Activity Diagram

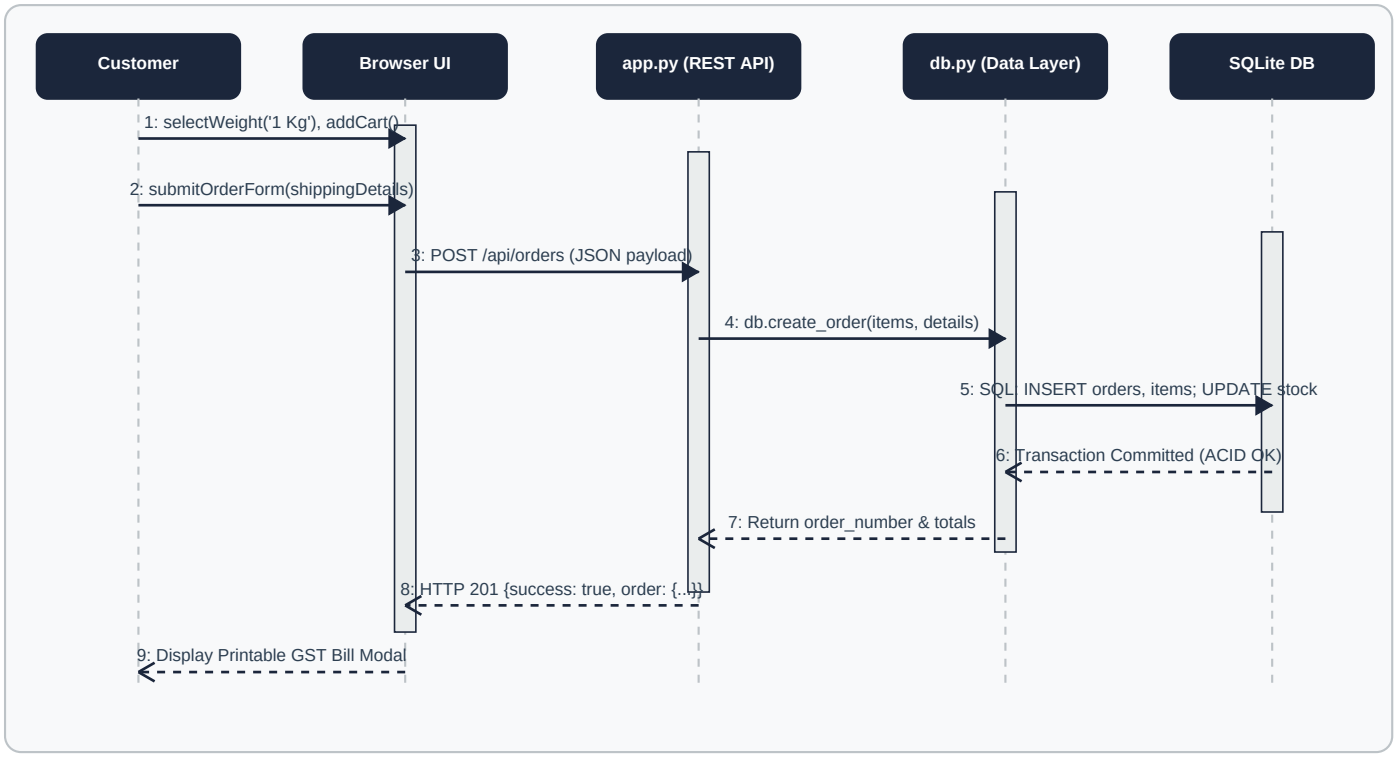
The Activity Diagram models the step-by-step business process flow for product selection, stock verification, tax calculation, and order processing across three swimlanes (Customer, Controller, Database).



Swimlane Execution & Transactional Flow:
* **Customer Swimlane:** Initiates selection, provides shipping credentials, and reviews final bill.
* **Controller Swimlane:** Performs validation, computes 2.5% CGST + 2.5% SGST, and routes payment confirmations.
* **Database Swimlane:** Executes atomic transactions. If stock is available, stock_qty is decremented and order is committed.

7. Sequence Diagram

The Sequence Diagram captures chronological interactions and synchronous/asynchronous message exchanges between actors, UI components, REST controllers, and database lifelines during order placement.



Step #	Sender -> Receiver	Message Signature / Operation	Description
1	Customer -> UI	selectWeight('1 Kg'), addCart()	User chooses packaging weight and updates browser cart.
2	Customer -> UI	submitOrderForm(shippingDetails)	Submits name, phone, address, and payment method.
3	UI -> app.py	POST /api/orders (JSON payload)	Asynchronous fetch API request with line items.
4	app.py -> db.py	create_order(items, details)	Invokes Python database transaction manager.
5	db.py -> SQLite	INSERT orders; UPDATE products	Executes atomic SQL transaction with stock deduction.
6-8	SQLite -> UI	HTTP 201 Created {order: {...}}	Returns confirmed order number, tax totals, and items.
9	UI -> Customer	renderBillModal(order)	Displays printable GST tax invoice with order confirmation.